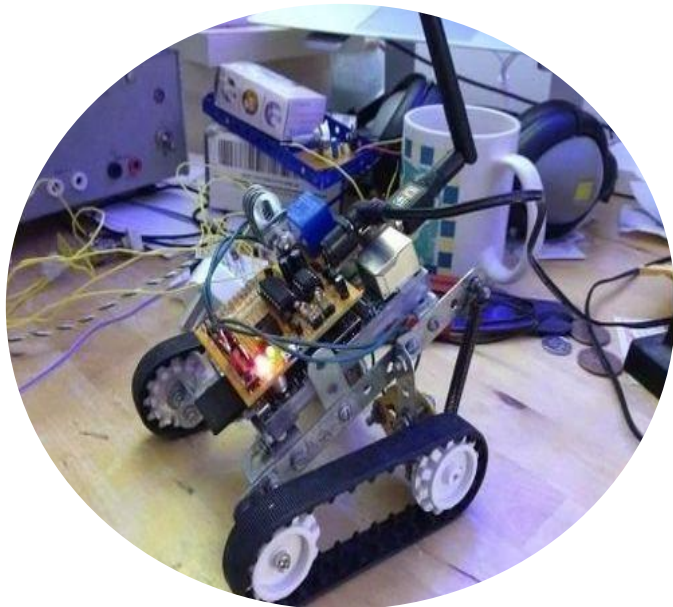
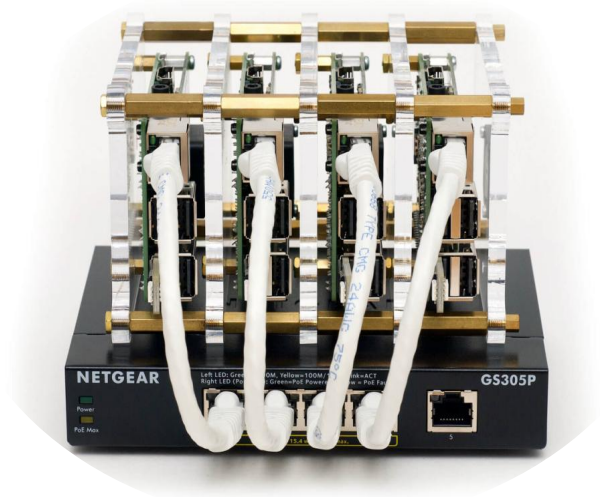




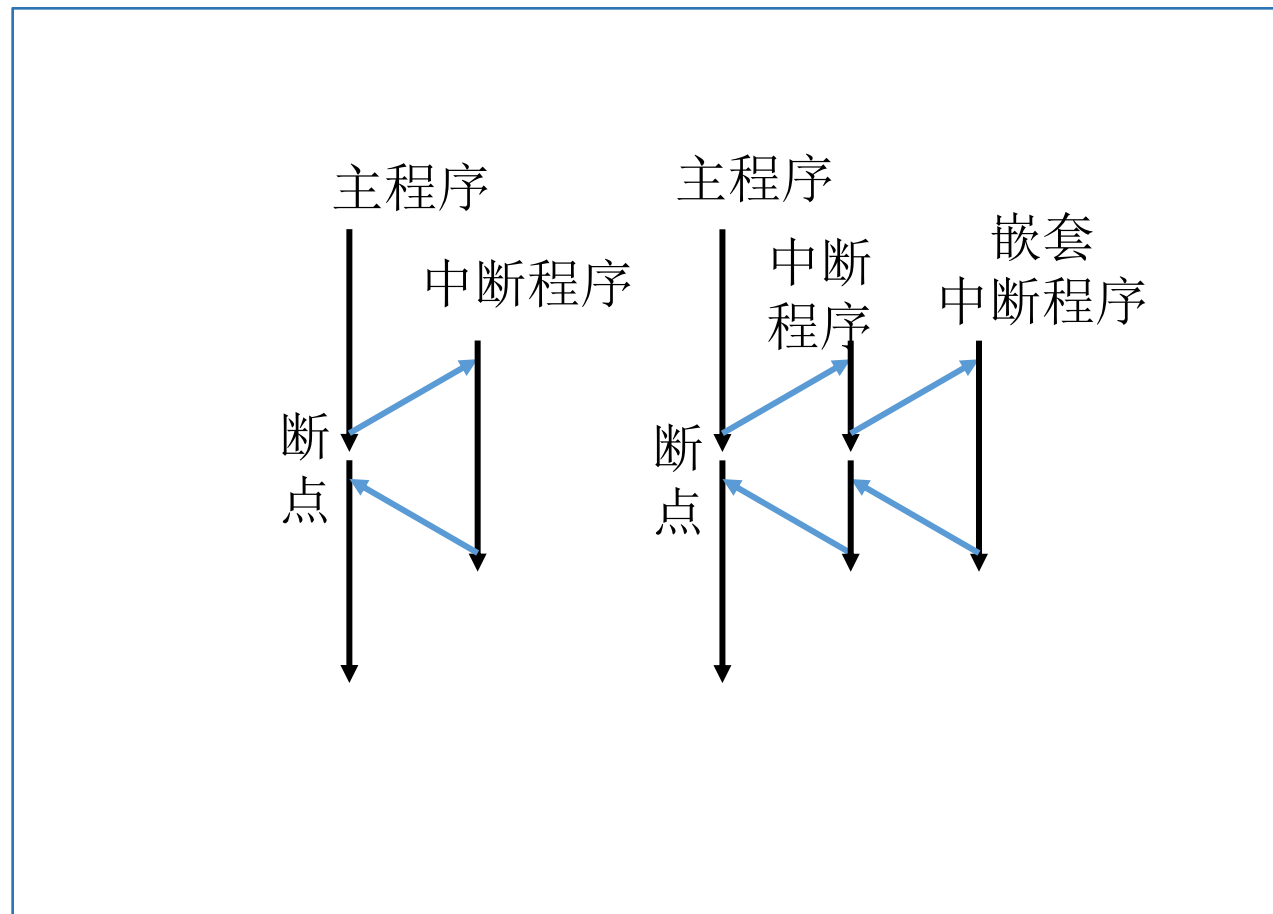
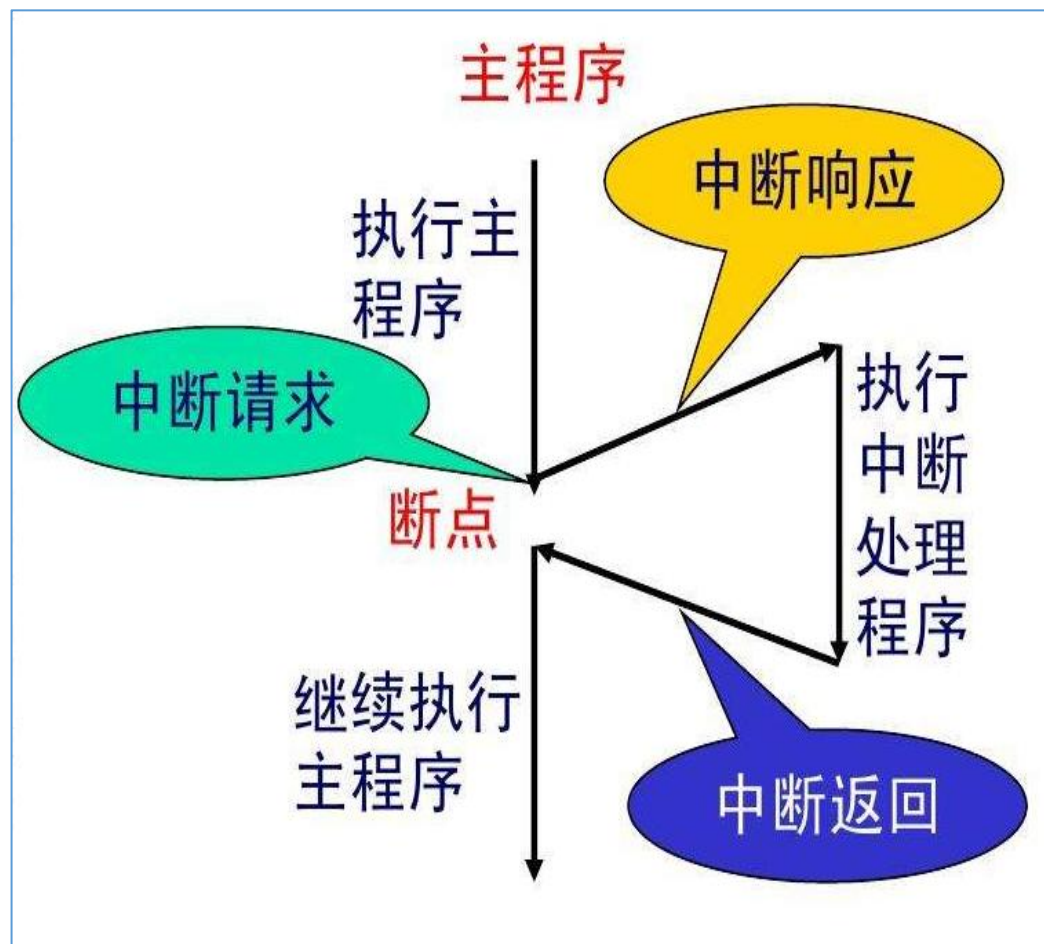
微机原理与微系统实验-树莓派



主讲老师：王小静
办公地点：慧园2栋411
open office hours：周三14:00-16:00

Lab 10: 关于中断

讲解STM32时对中断的记忆



```
// 设置中断组为0
NVIC_PriorityGroupConfig(NVIC_PriorityGroup_0);
```

```
// 中断优先级配置
static void BASIC_TIM_NVIC_Config(void)
{
    NVIC_InitTypeDef NVIC_InitStructure;

    // 设置中断来源
    NVIC_InitStructure.NVIC_IRQChannel = TIM6_IRQn ;
    // 设置主优先级为 0
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 0;
    // 设置抢占优先级为3
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 3;
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;
    NVIC_Init(&NVIC_InitStructure);
}
```

```
// 初始化定时器
TIM_TimeBaseInit(TIM6, &TIM_TimeBaseStructure);

// 3 清除计数器中断标志位
TIM_ClearFlag(TIM6, TIM_FLAG_Update);

// 开启计数器中断
TIM_ITConfig(TIM6, TIM_IT_Update, ENABLE);

//4 使能计数器
TIM_Cmd(TIM6, ENABLE);
```

```
void TIM6_IRQHandler(void)
{
    if ( TIM_GetITStatus( TIM6, TIM_IT_Update) != RESET )
    {
        time++;
        TIM_ClearITPendingBit(TIM6 , TIM_FLAG_Update);
    }
}
```

```
/******
芯片: STM32F103ZET6
实现功能: LED灯闪烁 (低电平点亮, 高电平熄灭), 通过基础定时器TIM6来实现1ms定时
引脚: PB5
*****/

#include "stm32f10x.h"
#include "led.h"
#include "timer_time.h"

uint16_t time=0;

int main(void)
{
    static uint8_t i=0;
    LED_GPIO_Config();
    // 设置中断组为0
    NVIC_PriorityGroupConfig(NVIC_PriorityGroup_0);

    BASIC_TIM_Init();

    while(1)
    {
        if( time == 1000 )
        {
            time = 0;
            switch(i)
            {
                case 0:LED_ON();i++;break;
                case 1:LED_OFF();i=0;break;
            }
        }
    }
}
```

为什么要有中断？

1. 解决 CPU 与外设的速度不匹配问题：CPU 的运算速度（GHz 级别）远高于外设（如硬盘、键盘、打印机，毫秒 / 秒级别）。
 - 轮询模式：CPU 需要不断循环检查外设“是否准备好”（比如反复问“键盘有没有按键？”），期间完全闲置，极大浪费算力；
 - 中断模式：外设准备好后主动“通知”CPU（触发中断），CPU 平时可执行核心任务，仅在收到中断时才处理外设请求，算力利用率大幅提升。

总结：让 CPU 主动去问”效率太低，必须反过来——让设备主动‘喊’CPU

2. 响应紧急 / 异常事件

系统中存在必须立即处理的事件（如电源故障、内存访问错误、除零运算），若依赖 CPU “按顺序处理”，可能导致系统崩溃或数据丢失。中断赋予这些事件更高优先级：无论 CPU 正在执行什么，都会立即暂停并处理紧急事件。

3. 实现多任务 / 并发的基础

现代操作系统的“分时复用”（多个程序看似同时运行），本质是通过时钟中断（固定时间间隔触发）实现：CPU 在不同任务间切换，每个任务执行一小段时间后，时钟中断触发，操作系统接管 CPU，切换到下一个任务，让用户感知到“并发”。

中断的执行流程

中断的处理是一套标准化流程，确保 CPU 能“有序暂停、处理、恢复”：

- 中断请求（IRQ）：外设 / 内部事件向 CPU 发送中断信号；
- 中断响应：CPU 完成当前指令后，暂停执行，保存当前上下文（程序计数器 PC、寄存器值等，通常压入栈）；
- 中断向量查找：CPU 根据中断号（唯一标识）查找“中断向量表”，找到对应的中断处理程序入口地址；
- 执行中断处理程序：CPU 跳转到处理程序，处理具体事件（如读取键盘按键值、修复内存错误）；
- 恢复上下文：处理完成后，从栈中恢复之前保存的寄存器和 PC 值；
- 中断返回：CPU 回到原来暂停的位置，继续执行原程序。

树莓派中断控制程序实例

树莓派官方 / 社区为了降低用户态开发门槛，把复杂的中断底层逻辑（如 **GPIO** 中断配置、中断监听、优先级处理）封装成了回调函数（**callback**）接口。

树莓派中断函数

➤ **add_event_detect(channel, edge, callback=None, bouncetime=None)**

Channel: 需要检测状态变化的GPIO引脚

Edge: 指定要监测的边沿类型，可以是GPIO.RISING, GPIO.FALLING, 或者GPIO.BOTH

Callback: 可选参数，指定状态变化时需要执行的回调函数

Bouncetime: 可选参数，用于消除抖动效应，指定两次状态变化之间的最小间隔(单位为ms)

-->异步非阻塞（程序继续跑，中断触发后回调），CPU占用极低

➤ **GPIO.wait_for_edge(channel, GPIO.RISING)**

主要用于在检测到边缘前阻止程序运行(占用CPU资源少)

-->同步阻塞（程序暂停等中断），CPU占用极低（%0 级别）

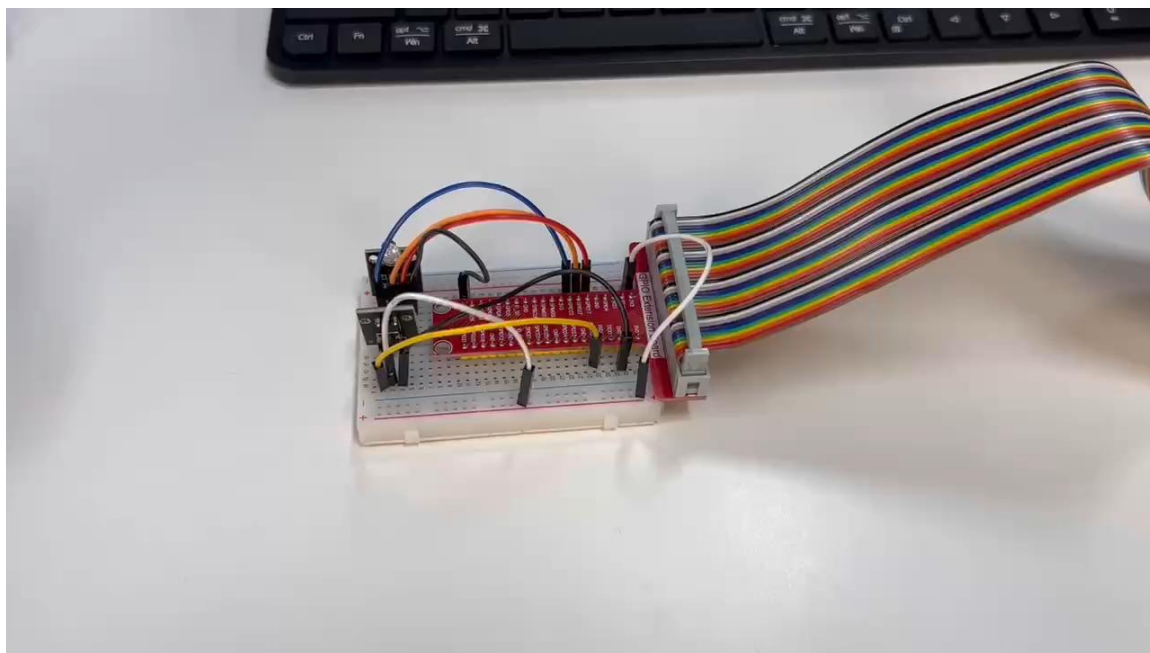


There are 2 kind of interrupts:

- RISING: when the state goes from LOW to HIGH.
- FALLING: when the state goes from HIGH to LOW.

树莓派中断控制程序实例

- 使用中断函数控制流水灯状态
- 调用callback函数 – 监视程序运行并及时中断



```
import RPi.GPIO as GPIO
import time

# GPIO引脚配置
LED_PINS = [17, 27, 22, 5] # 流水灯的LED引脚
BUTTON_PIN = 18 # 按键引脚

# 全局变量
running = False # 流水灯运行状态

# 初始化GPIO
GPIO.setmode(GPIO.BCM)
GPIO.setup(LED_PINS, GPIO.OUT) # 配置LED引脚为输出
GPIO.setup(BUTTON_PIN, GPIO.IN, pull_up_down=GPIO.PUD_UP) # 按键使用上拉电阻

# 定义按键中断回调函数
def button_callback(channel):
    global running
    running = not running # 切换运行状态
    print(f"Running state changed to: {'ON' if running else 'OFF'}")

# 设置按键中断
GPIO.add_event_detect(BUTTON_PIN, GPIO.FALLING, callback=button_callback, bouncetime=200)

print("Press the button to control the LED flow. Press Ctrl+C to exit.")

try:
    while True:
        if running:
            # 流水灯逻辑
            for pin in LED_PINS:
                GPIO.output(pin, GPIO.HIGH) # 点亮当前LED
                time.sleep(0.2) # 延迟
                GPIO.output(pin, GPIO.LOW) # 熄灭当前LED
            else:
                time.sleep(0.1) # 按键暂停时降低CPU占用率
except KeyboardInterrupt:
    print("Exiting program.")
finally:
    GPIO.cleanup() # 清理GPIO设置
```


实验要求

完成音乐播放中断功能：在音乐正常播放时按下按键，音乐播放中断，LED灯亮起，待LED灯闪烁几秒后自动继续播放之前的音乐。

